02 · Retries and Outcome Classification

When is it safe to try again? Classify every result precisely, retry only when provably safe, and verify otherwise.

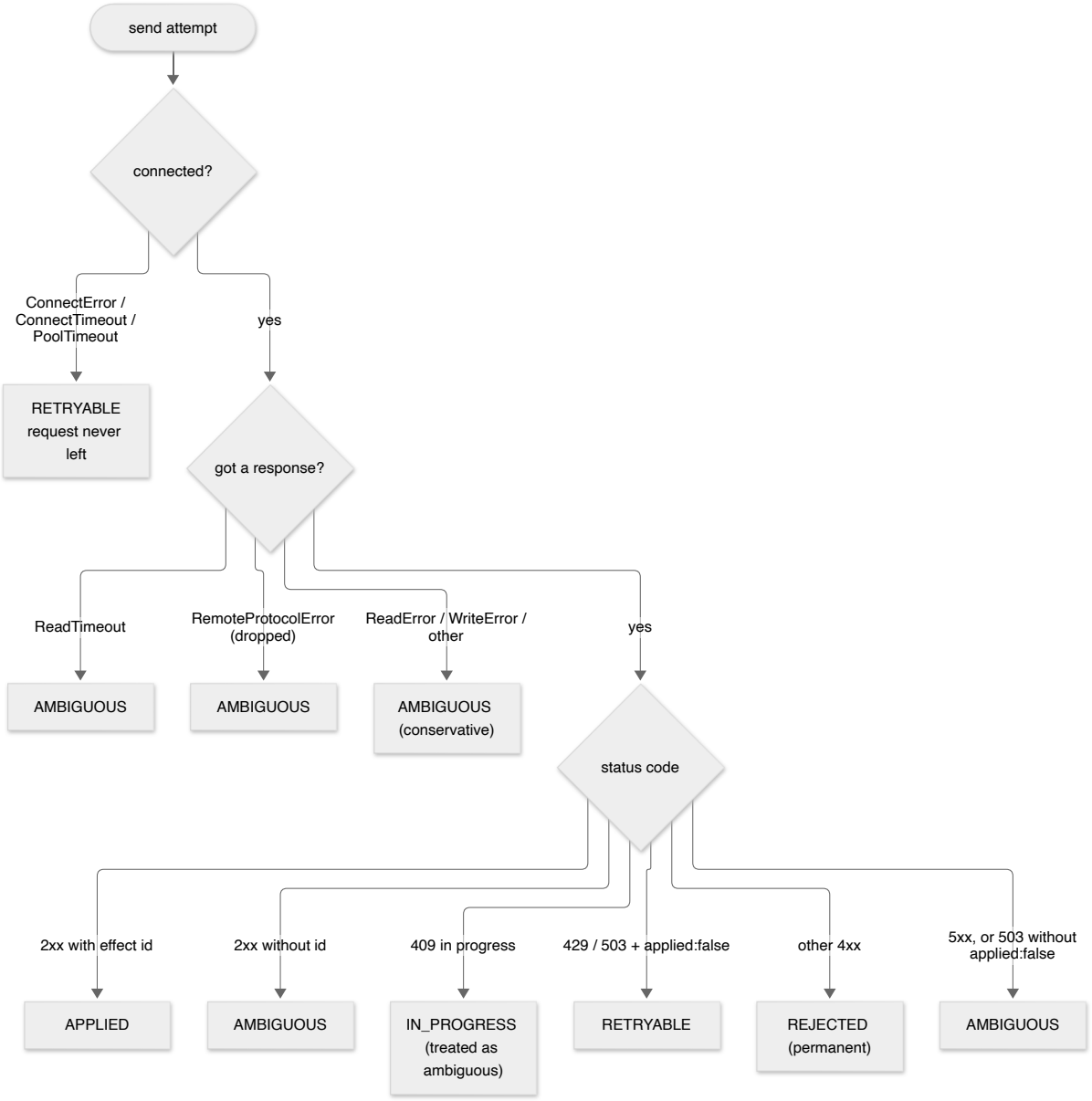
The problem

"Retry on error" is the most common reliability advice and, for side effects, the most dangerous. A retry is only harmless if the previous attempt **did not apply the effect**, or if the receiver **will deduplicate** the retry. Most errors leave you unsure which of those holds. So before deciding *whether* to retry, OneShot answers a sharper question:

> **After this attempt, is it possible that the effect was applied?**

Step 1: classify the result

`oneshot/provider_client.py` maps every possible HTTP result and exception to exactly one `Outcome`:



The guiding rule is that **when unsure, say AMBIGUOUS.** A wrong AMBIGUOUS costs one extra lookup or a deduplicated retry. A wrong RETRYABLE can cost a duplicate charge. Some examples of that rule at work:

- **Connection refused is safe.** If TCP never connected, no byte of the request reached the provider.
- **A 500 is not safe.** The server may have committed the charge and *then* crashed while building the response. The `error_after_apply` fault mode in the fake provider does exactly this (see doc 07).
- **A 503 with no body is not safe either.** A load balancer may return 503 after the backend did the work. Only the provider's explicit contract (`"applied": false`) makes a 503 provably harmless.

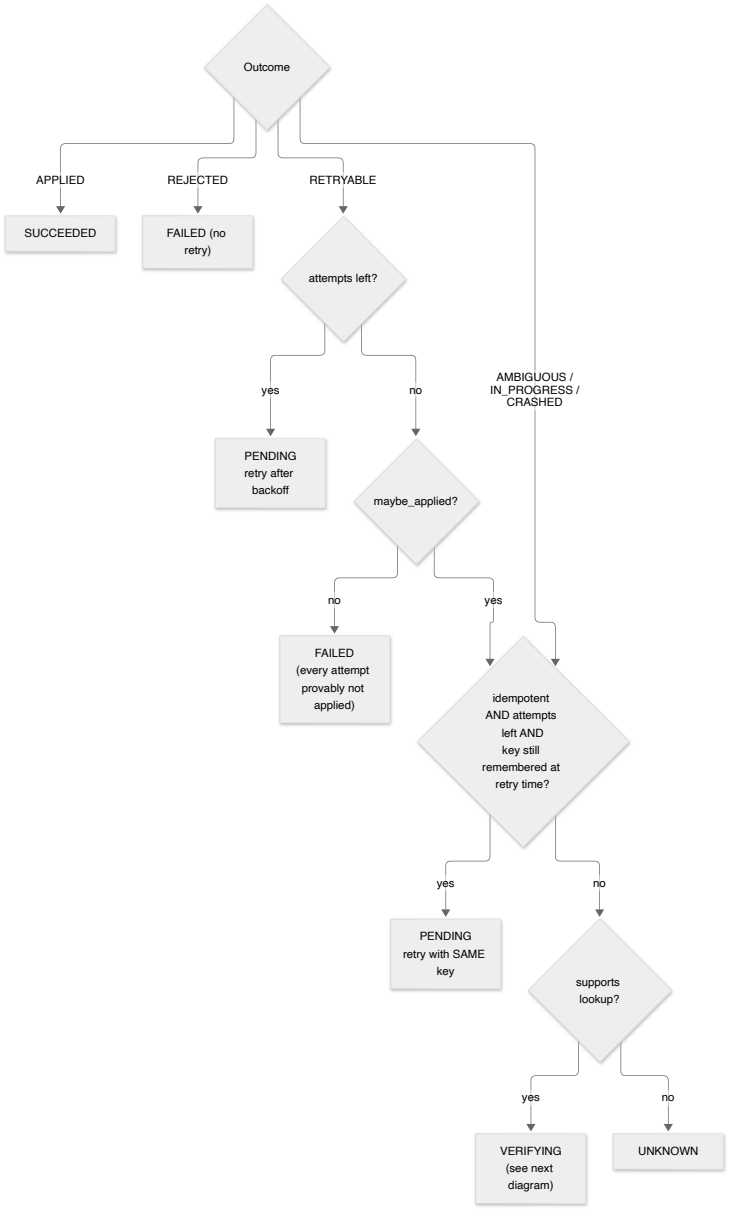
> These mappings come from the fake provider's API contract. Against a real provider you must read its documentation and redo this table. It is the most important piece of integration work, and the one most often skipped.

Step 2: decide, using the provider's capabilities

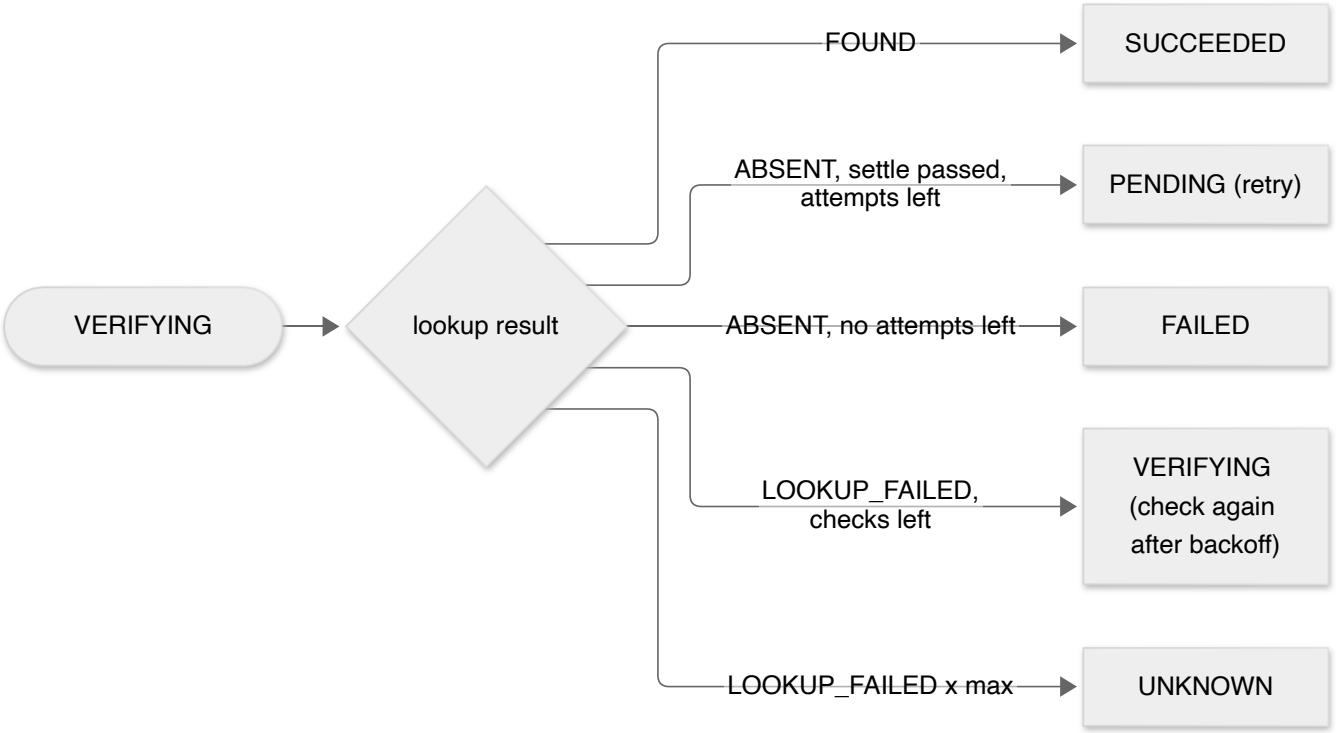
`oneshot/policy.py` holds pure functions with no I/O. Every action type declares two capabilities in `oneshot/action_types.py`:

Action type	`idempotent` (provider dedups by key)	`supports_lookup` (provider can be asked)
`payment.charge` → `POST /v1/charges`	yes	yes
`message.send` → `POST /v1/messages`	**no** (legacy endpoint)	yes

After a send attempt (`after_send`, `after_ambiguous`):



After a verification lookup (`after_verify`), run no earlier than `last_sent_at + provider_settle_seconds`:



The functions behind the diagram are `after_send`, `after_ambiguous`, `after_verify`, `before_send` and `same_key_retry_is_safe`, all in `oneshot/policy.py`.

The retention-window rule (and the bug it fixed)

Providers do not remember idempotency keys forever. The fake provider forgets them after `--idempotency-ttl` seconds (default 24 h, like many payment APIs). A same-key retry that arrives *after* the provider forgot the key is processed as a brand-new request, which means a duplicate. So `same_key_retry_is_safe()` requires:

```
(time of the retry) − first_sent_at  <  provider_key_retention_seconds − key_retention_margin_seconds
```

`first_sent_at` is written *before* the first request left, so it can only under-estimate the key's age, never over-estimate it.

The first version of the policy evaluated this rule at **decision time** (`now`), but the retry is sent at `now + backoff`. While writing the scenario `test_same_key_retry_is_not_attempted_once_provider_may_have_forgotten_the_key`, it became clear that a retry judged safe "now" could land after expiry. The fix (commit "*fix(policy): judge same-key retry safety at send time*") evaluates safety at `now + delay`. It also adds `before_send()`, a final check made immediately before any request leaves. That check covers a worker that was down for hours and picks up a retry scheduled long ago. It is defence in depth: the scheduler and the sender each verify safety independently.

Verification and the settle-window assumption

For a non-idempotent endpoint, an ambiguous outcome leads to `VERIFYING`. OneShot asks the provider `GET /v1/effects?reference=<action id>`, but **not immediately**. Right after a timeout the original request may still be executing, so "not found" would prove nothing. The check waits until `last_sent_at + provider_settle_seconds`.

> **Explicit assumption:** once sent, the provider finishes a request (or abandons it forever) within `provider_settle_seconds`. If the provider can be slower than that, "absent after the settle window" is wrong, and the retry that follows creates a duplicate. The demo scenario `slow-provider-duplicate` shows this happening, and reconciliation catches it (doc 03, doc 05).

`IN_PROGRESS` (HTTP 409, "a request with this key is still running") is handled as ambiguous, but the next retry waits at least the settle window. Retrying faster would only collect more 409s and burn the attempt budget.

Step 3: wait sensibly (exponential backoff with full jitter)

`backoff_delay(n)` returns a delay drawn uniformly from `[0, min(max, base × 2^(n-1))]`. With the defaults (`base = 0.5 s`, `max = 30 s`):

attempt that just failed	1	2	3	4	5	6	7+
ceiling (s)	0.5	1	2	4	8	16	30

Exponential growth gives a struggling provider room to recover. *Jitter* spreads retries from many clients over time, so they do not all hammer the provider at the same instant after an outage (the "thundering herd"). This is the "full jitter" strategy from the AWS Architecture Blog post *Exponential Backoff and Jitter* (2015). The demo turns jitter off (`backoff_jitter=False`) only so the printed delays are predictable.

Observed: classification tests

```
$ python -m pytest -v tests/test_provider_client.py
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

====================================== test session starts ======================================
collecting ... collected 19 items

tests/test_provider_client.py::test_response_classification[response0-APPLIED] PASSED        [  5%]
tests/test_provider_client.py::test_response_classification[response1-AMBIGUOUS] PASSED      [ 10%]
tests/test_provider_client.py::test_response_classification[response2-REJECTED] PASSED       [ 15%]
tests/test_provider_client.py::test_response_classification[response3-REJECTED] PASSED       [ 21%]
tests/test_provider_client.py::test_response_classification[response4-REJECTED] PASSED       [ 26%]
tests/test_provider_client.py::test_response_classification[response5-IN_PROGRESS] PASSED    [ 31%]
tests/test_provider_client.py::test_response_classification[response6-RETRYABLE] PASSED      [ 36%]
tests/test_provider_client.py::test_response_classification[response7-RETRYABLE] PASSED      [ 42%]
tests/test_provider_client.py::test_response_classification[response8-AMBIGUOUS] PASSED      [ 47%]
tests/test_provider_client.py::test_response_classification[response9-AMBIGUOUS] PASSED      [ 52%]
tests/test_provider_client.py::test_response_classification[response10-AMBIGUOUS] PASSED     [ 57%]
tests/test_provider_client.py::test_response_classification[response11-AMBIGUOUS] PASSED     [ 63%]
tests/test_provider_client.py::test_replay_is_detected PASSED                                [ 68%]
tests/test_provider_client.py::test_exception_classification[exc0-RETRYABLE] PASSED          [ 73%]
tests/test_provider_client.py::test_exception_classification[exc1-RETRYABLE] PASSED          [ 78%]
tests/test_provider_client.py::test_exception_classification[exc2-AMBIGUOUS] PASSED          [ 84%]
tests/test_provider_client.py::test_exception_classification[exc3-AMBIGUOUS] PASSED          [ 89%]
tests/test_provider_client.py::test_exception_classification[exc4-AMBIGUOUS] PASSED          [ 94%]
tests/test_provider_client.py::test_exception_classification[exc5-AMBIGUOUS] PASSED          [100%]

====================================== 19 passed in 0.03s =======================================
```

Observed: policy tests

```
$ python -m pytest -v tests/test_policy.py
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

====================================== test session starts ======================================
collecting ... collected 22 items

tests/test_policy.py::test_applied_succeeds PASSED                                        [  4%]
tests/test_policy.py::test_permanent_rejection_fails_without_retry PASSED                 [  9%]
tests/test_policy.py::test_provably_not_applied_is_retried_for_every_action_type[payment.charge] PASSED [
13%]
tests/test_policy.py::test_provably_not_applied_is_retried_for_every_action_type[message.send] PASSED [
18%]
tests/test_policy.py::test_provably_not_applied_is_retried_for_every_action_type[test.blind] PASSED [
22%]
tests/test_policy.py::test_retries_exhausted_with_only_clean_failures_is_failed PASSED    [ 27%]
tests/test_policy.py::test_retries_exhausted_after_an_ambiguous_attempt_is_never_failed PASSED [ 31%]
tests/test_policy.py::test_ambiguous_idempotent_retries_with_same_key PASSED              [ 36%]
tests/test_policy.py::test_ambiguous_non_idempotent_verifies_instead_of_retrying PASSED   [ 40%]
tests/test_policy.py::test_ambiguous_without_idempotency_or_lookup_is_unknown PASSED      [ 45%]
tests/test_policy.py::test_in_progress_and_crash_are_treated_as_ambiguous PASSED          [ 50%]
tests/test_policy.py::test_same_key_retry_stops_before_provider_forgets_the_key PASSED    [ 54%]
tests/test_policy.py::test_verify_found_means_succeeded PASSED                            [ 59%]
tests/test_policy.py::test_verify_absent_after_settle_retries PASSED                      [ 63%]
tests/test_policy.py::test_verify_absent_before_settle_waits PASSED                       [ 68%]
tests/test_policy.py::test_verify_failures_end_in_unknown PASSED                          [ 72%]
tests/test_policy.py::test_verify_found_twice_flags_duplicate PASSED                      [ 77%]
tests/test_policy.py::test_backoff_grows_exponentially_and_is_capped PASSED               [ 81%]
tests/test_policy.py::test_jittered_backoff_stays_within_bounds PASSED                    [ 86%]
tests/test_policy.py::test_retry_scheduled_past_retention_window_is_not_a_blind_retry PASSED [ 90%]
tests/test_policy.py::test_pre_send_guard_cancels_a_late_retry PASSED                     [ 95%]
tests/test_policy.py::test_in_progress_waits_at_least_the_settle_window PASSED            [100%]

====================================== 22 passed in 0.01s =======================================
```

Observed: transient failure, retried with backoff (demo)

```
$ python -m oneshot demo transient-retry
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

──────────────────────────────────────────── OneShot demo ────────────────────────────────────────────
Each scenario uses a fresh provider process with its own ledger, a fresh OneShot database, and real
worker processes. Faults are injected at the provider (or as failpoints in the worker).
Working directory: $TMPDIR/oneshot-demo-ayz4fwnh

1/1  transient-retry: Provider is briefly unavailable (503 twice) ──────────────────────────────────
  > fault armed at the provider: unavailable
  > agent POST /actions Idempotency-Key=agent-order-1002 -> HTTP 201 act_8daf37ee9702: charge
cus_ada 42.00 USD
  > start worker process
  > worker process finished: every action is SUCCEEDED, FAILED or UNKNOWN (exit 0)
  OneShot's record (event timeline):
  act_8daf37ee9702  charge cus_ada 42.00 USD  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87175, intent recorded before
sending
    +  0.07s IN_FLIGHT -> PENDING  HTTP 503 unavailable (applied=false); definitely not applied ->
retry in 0.30s (attempt 2/4)
    +  0.42s PENDING -> IN_FLIGHT  attempt 2: claimed by worker-87175, intent recorded before
sending
    +  0.43s IN_FLIGHT -> PENDING  HTTP 503 unavailable (applied=false); definitely not applied ->
retry in 0.60s (attempt 3/4)
    +  1.08s PENDING -> IN_FLIGHT  attempt 3: claimed by worker-87175, intent recorded before
sending
    +  1.08s IN_FLIGHT -> SUCCEEDED  provider confirmed: applied as eff_00001
```

```
  Provider's side of the story (its log):
    POST /v1/charges ref=act_8daf37ee9702 key=act_8daf37ee9702 mode=unavailable -> 503 unavailable
(not applied)
    POST /v1/charges ref=act_8daf37ee9702 key=act_8daf37ee9702 mode=unavailable -> 503 unavailable
(not applied)
    POST /v1/charges ref=act_8daf37ee9702 key=act_8daf37ee9702 mode=success -> APPLIED eff_00001,
responding 201
  Provider's independent ledger:
    act_8daf37ee9702: 1 effect(s) ['eff_00001']
  [PASS] act_8daf37ee9702 is SUCCEEDED and provider ledger has 1 effect(s) (observed: SUCCEEDED, 1)
  [PASS] took exactly 3 send attempts
  Lesson: A 503 with applied=false PROVES nothing happened, so retrying is safe for any action type.
Backoff (0.3s, 0.6s, ...) gives the provider room to recover.

Scorecard (every expectation checked against
the provider's real ledger)
┌──────────────────┬──────────┬──────┬──────────┐
│ scenario         │ checks   │ time │ result   │
├──────────────────┼──────────┼──────┼──────────┤
│ transient-retry  │      2/2 │ 1.2s │ PASS     │
└──────────────────┴──────────┴──────┴──────────┘
```

Observed: an ambiguous timeout resolved by verification (demo)

```
$ python -m oneshot demo timeout-message
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

──────────────────────────────────────────── OneShot demo ────────────────────────────────────────────
Each scenario uses a fresh provider process with its own ledger, a fresh OneShot database, and real
worker processes. Faults are injected at the provider (or as failpoints in the worker).
Working directory: $TMPDIR/oneshot-demo-4zfa6zjf

1/1  timeout-message: Provider hangs and never applies the message ──────────────────────────────────
  > fault armed at the provider: timeout on /v1/messages
  > agent POST /actions Idempotency-Key=agent-notify-2002 -> HTTP 201 act_8fd5c87d709c: message to
+15550100: 'Your order has shipped'
  > start worker process
  > worker process finished: every action is SUCCEEDED, FAILED or UNKNOWN (exit 0)
  OneShot's record (event timeline):
  act_8fd5c87d709c  message to +15550100: 'Your order has shipped'  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87190, intent recorded before
sending
    +  1.07s IN_FLIGHT -> VERIFYING  timed out waiting for the response; outcome unknown and
provider does not deduplicate -> check provider's record at settle time (+0.50s) before deciding
    +  1.63s VERIFYING -> PENDING  no record 1.5s after the last send -> treated as NOT applied
(assumes provider settles within that window) -> safe to retry
    +  1.63s PENDING -> IN_FLIGHT  attempt 2: claimed by worker-87190, intent recorded before
sending
    +  1.64s IN_FLIGHT -> SUCCEEDED  provider confirmed: applied as eff_00001
  Provider's side of the story (its log):
    POST /v1/messages ref=act_8fd5c87d709c key=- mode=timeout -> hanging 1.2s, will NOT apply, then
drop connection
    POST /v1/messages ref=act_8fd5c87d709c key=- mode=success -> APPLIED eff_00001, responding 201
  Provider's independent ledger:
    act_8fd5c87d709c: 1 effect(s) ['eff_00001']
  [PASS] act_8fd5c87d709c is SUCCEEDED and provider ledger has 1 effect(s) (observed: SUCCEEDED, 1)
  Lesson: From the outside this timeout looks IDENTICAL to the lost response above. Verification is
what tells them apart: here the record is absent after the settle window, so a resend is safe.

Scorecard (every expectation checked against
the provider's real ledger)
┌──────────────────┬──────────┬──────┬──────────┐
│ scenario         │ checks   │ time │ result   │
├──────────────────┼──────────┼──────┼──────────┤
│ timeout-message  │      1/1 │ 1.7s │ PASS     │
└──────────────────┴──────────┴──────┴──────────┘
```

Protects against: duplicates from blind retries after timeouts, dropped connections and 5xx; giving up on transient failures; hammering a recovering provider; retrying after the provider has forgotten the key (when retention is configured correctly); stale retries fired by a worker that was down for a long time.

Cannot protect against: a provider that violates its own contract (for example a 4xx after applying); a provider slower than the settle window for non-idempotent endpoints; a retention window configured longer than the provider's real one (`test_misconfigured_retention_window_causes_a_duplicate` proves the duplicate happens); retry storms from many OneShot instances during a long outage (no circuit breaker yet; see ROADMAP).

Design decisions and alternatives

- **Classification separate from decision.** `provider_client` knows HTTP; `policy` knows safety. Changing providers means changing one; changing the safety rules means changing the other.
- **Pure functions with an injected clock.** `tests/test_policy.py` checks every branch without sleeping or starting servers. It runs in milliseconds.
- **Rejected: generic retry libraries** (for example *tenacity*, or `urllib3.Retry`). They decide on the type of exception, not on whether the effect may have applied. `urllib3.Retry`, for instance, does not retry POST by default *precisely because* POST is not idempotent.
- **Rejected: decorrelated jitter / equal jitter.** These are fine alternatives; full jitter is the simplest to explain and performs well in the AWS analysis.

Self-check questions

1. Why is "connection refused" RETRYABLE but "connection reset after sending" AMBIGUOUS?
2. A `payment.charge` times out. Retention is 24 h, `first_sent_at` was 2 minutes ago. What happens next, and why is it safe?
3. The same timeout happens for `message.send`. What happens next? When exactly is the provider checked?
4. Why does the policy evaluate key retention at `now + delay` rather than `now`?
5. Why is jitter useful even though it makes individual retries *less* predictable?

Answers. (1) Refused means no request ever reached the server; a reset after sending means the server may have processed it. (2) PENDING with a same-key retry after backoff; the provider either replays the stored result or processes it for the first time, never twice. (3) VERIFYING; the lookup runs at `last_sent_at + provider_settle_seconds`; FOUND → SUCCEEDED, ABSENT → retry. (4) The retry is *sent* later; what matters is whether the provider still remembers the key when it arrives. (5) It de-synchronises many clients, so a recovering provider sees a smooth trickle instead of synchronized spikes.